Guía del notebook `1_03_2_regla_de_aprendizaje`: el perceptrón aprende

Puntos: 4 · **Tiempo:** 25–35 min · **Clase:** 2 · **Archivo:** `notebooks/1_03_2_regla_de_aprendizaje.ipynb`
Teoría completa: Guía de la Clase 2, sección B4.

¿Qué vas a aprender?

- Que el perceptrón puede **aprender solo** sus pesos a partir de ejemplos.
- A medir qué tanto se equivoca (la **pérdida**) y a ver cómo baja con el entrenamiento.
- A dibujar la **frontera de decisión** (la recta que separa las clases).

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_03_2_regla_de_aprendizaje.ipynb` → kernel **nni**.
   Usa la carpeta del repo, porque el notebook importa el paquete `nni` que está ahí.
3. Ejecuta las celdas en orden con **Shift + Enter**.

> **En Colab:** primero ejecuta en una celda nueva `!git clone https://github.com/datahuba/curso-redes-neuronales` y `%cd curso-redes-neuronales/notebooks`.

La idea en 2 minutos

Regla de Rosenblatt, aplicada a cada ejemplo:

$$W \leftarrow W + \eta \cdot (y - \hat{y}) \cdot X \quad y \quad b \leftarrow b + \eta \cdot (y - \hat{y})$$

- **error = $y - \hat{y}$**: 0 si acertó, +1 o −1 si se equivocó.
- **η** (tasa de aprendizaje): el tamaño de la corrección.
- **Época:** una pasada por todos los ejemplos.
- **SAE** (suma de errores absolutos): cuántos ejemplos se equivoca en total. Con etiquetas 0/1, es exactamente el número de errores.

Paso a paso

Celdas 1–2: datos sintéticos

Cargan las librerías y generan **400 puntos** de dos clases (rojos = 0, azules = 1), que se dibujan.

Celdas 3–6: la clase `Perceptron`

Ya está hecha. Empieza con pesos en cero. Ejecútalas.

Celda `TODO`: inicialización (1 punto)

Asigna pesos aleatorios a `clasificador.W` (dos números) y un sesgo a `clasificador.b`.

- Pista: `np.random.normal(loc=0, scale=1, size=(2,))` genera dos números al azar alrededor de 0.
- El sesgo puede empezar en 0.0.
- Para obtener siempre los mismos números (y poder compararlos), escribe antes `np.random.seed(0)`.

✅ **Comprobación:** con `np.random.seed(0)` los pesos son ≈ **[1.764, 0.400]**.

Celda `TODO: perdidaSAE` (1 punto)

Suma de los valores absolutos de (y_true − y_pred).

- Pistas: `np.abs` y `np.sum`.

✅ **Comprobación:** con la semilla 0, la pérdida inicial es **93** (se equivoca en 93 de 400 puntos). Con otra semilla sale otro número; lo importante es que sea un entero entre 0 y 400.

Celda: gráfica del clasificador sin entrenar

Dibuja la frontera con pesos al azar. Normalmente separa mal los colores.

> ⚠️ Con scikit-learn 1.9 o más nuevo, esta gráfica exige el argumento `n_classes=2`. La copia del repositorio ya lo trae. Si usas una versión vieja y sale `unexpected keyword argument 'n_classes'`, bórralo.

Celda: `ReglaDeAprendizaje` (1 punto)

Completa las dos líneas `None` con la regla de arriba:

- los pesos nuevos son los pesos actuales más tasa · error · `xi`;
- el sesgo nuevo es el sesgo actual más tasa · error.

Celdas de entrenamiento

`n_epocas = 3` suele quedarse corto. La gráfica muestra la pérdida por época.

Celda `TODO (2 puntos)`: ajustar hiperparámetros

Sube `n_epocas` (por ejemplo a 20–50) y prueba `tasa_aprendizaje` entre 0.01 y 0.5. Después vuelve a ejecutar las celdas de entrenamiento y de la gráfica final.

✅ **Comprobación** (semilla 0, tasa 0.1, 20 épocas):

- la pérdida baja de **93** a unos **37**;
- tras la última actualización, el notebook queda en **34 errores de 400**;
- la frontera separa la mayoría de los puntos rojos y azules.

No tiene que ser perfecto: los datos se mezclan un poco y una recta no puede separarlos al 100 %.

Para pensar

- ¿Por qué la pérdida a veces **sube** de una época a otra? → La regla corrige ejemplo por ejemplo; arreglar uno puede descomponer otro.
- ¿Llegará alguna vez a 0? → Solo si los datos son linealmente separables.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: ... missing ... 'n_classes'`	scikit-learn nuevo	Agrega `n_classes=2,` dentro de `DecisionBoundaryDisplay(...)`
`ModuleNotFoundError: No module named 'nni'`	Notebook abierto fuera del repo	Abre la carpeta `curso-redes-neuronales` completa
La pérdida no cambia	`perceptron.W` sigue en `None`, o no ejecutaste la celda de la regla	Completa la regla y ejecútala de nuevo
Resultados distintos cada vez	Pesos iniciales al azar	Usa `np.random.seed(0)` antes de inicializar